ParkYa — Entrega 2: plan de trabajo

Para: **Camilo Badillo** (Dev 1) y **Moisés Carrillo** (Dev 2) · De: Jaime Granados (líder)

Fecha de entrega al profesor: _____ Fecha límite de sus PR: _____

1. Contexto

La Entrega 2 convierte el diseño en una base de datos real en PostgreSQL 16+: migraciones, datos de prueba, consultas de negocio, normalización e índices. **Ustedes dos construyen la base, los datos y las consultas**; el resto de la entrega lo cubro yo.

Antes de empezar, lean en el repositorio (rama `main`):

- `docs/esquema-relacional.md` — **la especificación**: cada tabla, columna, tipo, nulabilidad, PK, FK, política de borrado, CHECK, UNIQUE y EXCLUDE. Es lo que hay que implementar, tal cual.
- `docs/correcciones-entrega-1.md` — qué cambió frente a la Entrega 1 y por qué (en la sustentación les pueden preguntar).
- `diagramas/eer-chen-parkya-entrega2.png` — el diagrama EER corregido.
- Los `README.md` de `migraciones/`, `seeds/`, `consultas/` y `scripts/`.

Importante: la guía del curso castiga que un solo integrante haga todos los commits, y en la sustentación final hay preguntas individuales sobre cualquier parte. Cada uno sube su propio trabajo con su propia cuenta de GitHub y debe poder explicar lo que hizo el otro.

2. Reparto

	Camilo (Dev 1) — Esquema	Moisés (Dev 2) — Datos
Bloque A	Migraciones `001`–`003` (todas las tablas y restricciones) + `scripts/reconstruir.ps1`	`seeds/01_datos_base.sql` (datos coherentes para las consultas)
Bloque B	`004_indices_fk.sql` + `005_vistas.sql`	`seeds/02_volumen.sql` (miles de filas para medir índices)
Bloque C	Consultas 5 a 8 (vista, CTE, subconsulta)	Consultas 1 a 4 (joins de 3+ tablas, LEFT JOIN, GROUP BY + HAVING)
Bloque D	`docs/diccionario-de-datos.md` actualizado con lo implementado	Verificación a mano de las 8 consultas contra los datos semilla

Dependencia clave: Moisés no necesita esperar a Camilo para *escribir* los seeds (la especificación ya está en `esquema-relacional.md`), pero solo los puede *probar* cuando las migraciones 001–003 estén en `main`. Por eso Camilo debe abrir su primer PR lo antes posible.

3. Cronograma

Días	Camilo	Moisés
1 – 3	001, 002, 003 y reconstruir.ps1. **Primer PR.**	Escribir 01_datos_base.sql contra la especificación.
4	Corregir lo que salga de la revisión del PR.	Probar los seeds sobre la base reconstruida y corregir.
5 – 7	004, 005 y consultas 5–8.	02_volumen.sql y consultas 1–4.
8	Diccionario de datos actualizado. **Último PR.**	Verificación a mano de las 8 consultas. **Último PR.**

4. Camilo (Dev 1) — Esquema

Bloque A. Migraciones 001–003

- Reparto de tablas por archivo: ver la tabla de `migraciones/README.md`. `001` empieza con `CREATE EXTENSION IF NOT EXISTS btree_gist;` (lo necesitan los EXCLUDE de `tarifa` y `suscripcion`).
- Implementar **todas** las restricciones de la especificación, sin omitir ninguna: PK, FK con su `ON DELETE`, NOT NULL, UNIQUE, CHECK, EXCLUDE y los dos índices únicos parciales de `sesion`. Es el 30 % de la nota.
- Claves sustitutas con `GENERATED ALWAYS AS IDENTITY`.
- Todas las restricciones con nombre: `pk_`, `fk_`, `uq_`, `ck_`, `ex_` + tabla + detalle.
- Cada archivo empieza con un comentario: qué crea y de qué migración depende.

```
CREATE TABLE sesion (
    id_sesion       INTEGER GENERATED ALWAYS AS IDENTITY,
    placa           VARCHAR(10) NOT NULL,
    ...
    CONSTRAINT pk_sesion PRIMARY KEY (id_sesion),
    CONSTRAINT fk_sesion_vehiculo FOREIGN KEY (placa)
        REFERENCES vehiculo (placa) ON DELETE RESTRICT,
    CONSTRAINT ck_sesion_horas CHECK (hora_salida > hora_entrada)
);
CREATE UNIQUE INDEX uq_sesion_abierta_placa ON sesion (placa) WHERE hora_salida IS NULL;
```

scripts/reconstruir.ps1: borra la base `parkya` si existe, la crea, ejecuta `migraciones/*.sql` en orden de nombre y después `seeds/*.sql`. Debe usar `psql -v ON_ERROR_STOP=1` y detenerse en el primer error. Credenciales solo por variables de entorno (`PGHOST`, `PGUSER`, `PGPASSWORD`); **nunca** una contraseña escrita en el script.

Prueba antes del PR: ejecutar el script dos veces seguidas sobre una base vacía sin errores, e intentar a mano algunos INSERT inválidos (una sesión con salida antes de la entrada, dos tarifas que se cruzan, un pago sin sesión ni suscripción) para comprobar que la base los rechaza.

Bloque B. Índices de FK y vistas

- `004_indices_fk.sql`: los índices de la sección "Índices en FK de joins frecuentes" de `esquema-relacional.md`. PostgreSQL no indexa las FK automáticamente.
- `005_vistas.sql`: la vista `v_ocupacion_actual` (ver consulta 5).
- Recordatorio: una migración que ya está en `main` **no se edita**. Si hay que corregir algo, se crea la siguiente (`007_corrige_....sql`). El índice `006` lo reservo yo para el caso con EXPLAIN ANALYZE.

Bloque C. Consultas 5 a 8

#	Pregunta de negocio	Técnica exigida
5	¿Cuántos espacios activos, ocupados y libres tiene cada sede en este momento? (RF7). La vista se crea en `005_vistas.sql` y se consulta en el banco (por ejemplo, filtrando sedes con menos de 5 espacios libres).	**Vista creada y consultada**
6	¿Qué tarifa por hora se cobraba en cada sede y tipo de vehículo en una fecha pasada? (RF5)	**Subconsulta** (o rango con `daterange @>`)
7	¿Qué suscripciones vencen en los próximos 7 días, de qué cliente y vehículo, y cuántos días les quedan? (RF4, para ofrecer la renovación)	**CTE** (`WITH`)
8	De los vehículos que están parqueados ahora mismo, ¿cuáles están cubiertos por una suscripción activa y cuáles tendrán que pagar al salir? (RF2)	LEFT JOIN + condición de fechas

Bloque D. Diccionario de datos

Reescribir `docs/diccionario-de-datos.md` tabla por tabla con el esquema **realmente implementado**: columna, tipo, nulo, restricciones y significado de negocio. Si algo cambió al implementar, manda lo que está en las migraciones (y avisen para anotarlo en las correcciones).

5. Moisés (Dev 2) — Datos y consultas

Bloque A. seeds/01_datos_base.sql

- Datos **ficticios pero coherentes**. Nada de `cliente1`, ni nombres o documentos de personas reales (tampoco los nuestros).
- Cantidades orientativas: 3 sedes, 3 tipos de vehículo (carro, moto, camioneta), unos 20 espacios por sede en 2–3 pisos (algunos preferenciales y alguno en mantenimiento), 8–10 empleados, 25–30 clientes, 35–40 vehículos, planes mensual y trimestral por tipo, 10–15 suscripciones, 60–100 sesiones con sus pagos y 10–15 incidencias.
- Tarifas con **historial**: al menos un cambio de precio por sede (una tarifa cerrada con `vigencia_hasta` y la vigente con `NULL`), sin solapamientos.
- Fechas **relativas a hoy** (`CURRENT_DATE - 3`, `now() - interval '2 hours'`), no fechas fijas. Así las consultas de "ahora" y "próximos 7 días" siguen dando resultados el día de la entrega y de la sustentación.
- No escribir IDs a mano: resolver las FK con subconsultas por clave natural (nombre de sede, documento, placa).
- Placas con formato válido: carro `ABC123`, moto `ABC12D`; al menos un vehículo temporal `TMP0001`.

```
INSERT INTO empleado (id_sede, nombre, documento, cargo)
VALUES ((SELECT id_sede FROM sede WHERE nombre = 'ParkYa Centro'),
        'Laura Méndez', '52123456', 'operario');
```

Casos que los datos deben incluir a propósito (sin ellos, las consultas devuelven vacío):

- Empleados que **nunca** han reportado incidencias (consulta 3).
- Clientes ocasionales con muchas sesiones en el último mes y sin suscripción (consulta 4).
- Sesiones **abiertas** ahora mismo (`hora_salida` nula): algunas de vehículos con suscripción activa y otras sin ella (consultas 5 y 8).
- Suscripciones que vencen en los próximos 7 días, y alguna cancelada (consulta 7).
- Pagos de suscripciones además de pagos de sesiones; alguna sesión pagada en dos pagos.
- Una incidencia sin sesión (`id_sesion` nulo) y una sesión cerrada por un empleado.

Bloque B. seeds/02_volumen.sql

Generar con `generate_series` unas **50 000 sesiones** cerradas repartidas en los últimos 12 meses entre todas las sedes, espacios y vehículos, cada una con su pago. Sin este volumen, `EXPLAIN ANALYZE` no muestra diferencia entre usar o no un índice. Cuidado con las restricciones: horas de salida posteriores a la entrada, y sin dos sesiones abiertas del mismo vehículo o espacio (las de volumen deben ir todas cerradas).

Bloque C. Consultas 1 a 4

#	Pregunta de negocio	Técnica exigida
1	¿Cuánto se facturó por sede y por tipo de vehículo en un rango de fechas? (RF8)	**Join de 3+ tablas** + GROUP BY
2	Historial de un cliente: sus vehículos, en qué sede y espacio parqueó, cuánto tiempo, cuánto pagó y, si aplica, qué empleado cerró la sesión.	**Join de 3+ tablas**
3	¿Qué empleados no han reportado ninguna incidencia?	**LEFT JOIN "los que no tienen"** (`WHERE ... IS NULL`)

#	Pregunta de negocio	Técnica exigida
4	¿Qué clientes sin suscripción activa tuvieron más de N sesiones en los últimos 30 días? (candidatos para venderles un plan)	**GROUP BY + HAVING**

Ojo con los totales inflados (error típico según la guía): si se hace join con pagos e incidencias a la vez antes de agrupar, los montos se multiplican. Agregar primero (subconsulta o CTE) y unir después.

Bloque D. Verificación a mano

Para cada una de las 8 consultas (las 4 de Camilo también), contar o sumar a mano sobre `01_datos_base.sql` y anotar el resultado esperado en el comentario de la consulta. Si no coincide con lo que devuelve PostgreSQL, la consulta o los datos están mal.

6. Formato de cada consulta

Todas van en `consultas/banco_consultas.sql`. Camilo escribe las 5–8 y Moisés las 1–4 (para no pisarse: cada uno trabaja en su bloque del archivo y se resuelve el merge al final).

```
-- ============================================================
-- Consulta 3. ¿Qué empleados no han reportado ninguna incidencia?
-- Requisito: RF6  |  Técnica: LEFT JOIN "los que no tienen"
-- Verificación: en 01_datos_base.sql hay 9 empleados y 6 aparecen
--   en incidencia -> se esperan 3 filas. Resultado: 3 filas. OK
-- ============================================================
SELECT e.nombre, e.cargo, s.nombre AS sede
FROM empleado e
JOIN sede s ON s.id_sede = e.id_sede
LEFT JOIN incidencia i ON i.id_empleado = e.id_empleado
WHERE i.id_incidencia IS NULL
ORDER BY s.nombre, e.nombre;
```

7. Cómo trabajamos con Git

- Cada uno usa **su propia cuenta** de GitHub (ya les agrego como colaboradores) y configura su nombre y correo en Git antes del primer commit.
- Una rama por bloque, creada desde `main` actualizado. Commits pequeños y con mensaje claro (`feat: migracion 003 sesion, pago e incidencia`).
- Se integra por **pull request** hacia `main`; yo reviso y hago el merge. Nadie hace push directo a `main`.
- Antes de abrir el PR: reconstruir la base desde cero con el script y comprobar que no hay errores.

```
git clone https://github.com/JaimeGranados/PARKYA.git
git config user.name "Su Nombre"
git config user.email "su-correo@ejemplo.com"

git switch main
git pull
git switch -c feat/migraciones-ddl        (Moisés: feat/seeds-base)
git add migraciones/001_catalogos.sql
git commit -m "feat: migracion 001 catalogos"
git push -u origin feat/migraciones-ddl
(luego abrir el pull request en GitHub)
```

8. Lista de verificación final

Camilo

[ ] 001–003 crean todas las tablas con todas las restricciones de la especificación.

[ ] reconstruir.ps1 funciona dos veces seguidas sobre una base vacía, sin credenciales en el código.

[ ] 004 (índices de FK) y 005 (vista `v_ocupacion_actual`).

[ ] Consultas 5–8 comentadas y verificadas.

[ ] Diccionario de datos actualizado con lo implementado.

Moisés

[ ] 01_datos_base.sql: datos ficticios coherentes, fechas relativas, sin IDs a mano.

[ ] Incluye todos los casos especiales que necesitan las consultas.

[ ] 02_volumen.sql con unas 50 000 sesiones que no violan ninguna restricción.

[ ] Consultas 1–4 comentadas y verificadas.

[ ] Resultado esperado anotado a mano en las 8 consultas.

Dudas sobre el diseño o la especificación: escríbanme antes de cambiar algo por su cuenta. Si encuentran un error en `esquema-relacional.md`, mejor avisar que adivinar.